AdaGrad (Adaptive Gradient)

Optimizers in Deep Learning – Part 4

---

1. Intuition Behind AdaGrad

In all the previous optimizers (like SGD, Momentum, and NAG), we used **a single learning rate (η)** for all parameters.

However — different parameters may require **different learning rates**.

For example:

- Some weights may need **large updates** (if gradients are small).
- Others may need **tiny updates** (if gradients are large).

So, using one global learning rate can be inefficient.

That's where **AdaGrad (Adaptive Gradient)** comes in — it **automatically adjusts the learning rate** for each parameter individually during training.

---

2. The Core Idea - "Adaptive Learning Rate"

AdaGrad adapts the learning rate for each parameter **based on how frequently it's updated**:

- Parameters that have **large gradients** (change often) → get **smaller learning rates**
- Parameters that have **small gradients** (change rarely) → get **larger learning rates**

This allows the optimizer to take **bigger steps for infrequent features** and **smaller steps for frequent features** — especially useful in sparse data (like NLP or text data).

---

3. Example - Elongated Bowl Problem

Imagine your loss surface looks like a **long, curved valley (bowl)**:

- The slope is **steep** along one direction (vertical).
- The slope is **shallow** along the other (horizontal).

Normal gradient descent keeps oscillating because the same learning rate applies to both directions.

AdaGrad **fixes this** by reducing the learning rate **faster in steep directions** and **keeping it larger in flat directions**.

This helps move efficiently **toward the minimum** without zig-zagging.

---

4. Visual Representation

- Think of AdaGrad as a **smart driver** who adjusts speed for each road:
    - On a steep hill → slows down.
    - On a flat road → speeds up.

- The learning rate keeps adapting automatically for every parameter over time.

This dynamic control over the step size is what makes AdaGrad **adaptive** and **intelligent**.

---

5. Mathematical Formulation

Let's break it down step by step

Given:

- θ_t: model parameters at time t
- g_t: gradient at time t
- η: global learning rate
- G_t: sum of squares of past gradients (for each parameter)

Formulae:

1 Accumulate squared gradients:

$$G_t = G_{t-1} + g_t^2$$

2 Update parameters:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot g_t$$

Here:

- ϵ = small constant (like 1e-8) to prevent division by zero.

Meaning:

- As training progresses, (G_t) keeps growing.
- Thus, (1/sqrt{G_t) keeps **decreasing**, making the **effective learning rate smaller** over time.
- Parameters that get large gradients frequently will have their learning rate reduced quickly.
- Parameters that rarely get large gradients will still move more — adapting automatically.

---

6. Intuition — How Does It Behave?

Imagine tracking how often each weight is updated:

- If a weight often gets **large updates**, AdaGrad **slows it down**.

- If a weight rarely changes, AdaGrad **keeps its learning rate higher**.

So AdaGrad works like a **personalized learning rate manager** for each parameter.

---

7. Mathematical Intuition

The scaling factor (1/sqrt{G_t) acts like a **memory** of all previous gradients.
This ensures:

- Parameters with large past gradients (frequent updates) → learning rate becomes smaller.
- Parameters with small gradients (rare updates) → learning rate stays relatively higher.

This helps especially in problems like:

- Sparse data (text, word embeddings)
- Recommender systems
- High-dimensional datasets

---

8. Keras / TensorFlow Implementation

AdaGrad is built into most deep learning frameworks.

```
from tensorflow.keras.optimizers import Adagrad

optimizer = Adagrad(learning_rate=0.01, epsilon=1e-8)
```

TensorFlow automatically handles the gradient history and adaptive updates internally.

---

9. Disadvantage of AdaGrad

The main issue with AdaGrad is **aggressive decay of the learning rate**.
Because it **squares and accumulates gradients**, the denominator (\sqrt{G_t}) keeps **increasing forever**, which means the **learning rate keeps shrinking** — sometimes too much!
Eventually, the learning rate becomes so small that:

- The model **stops learning early**
- It **gets stuck** before reaching the global minimum

This problem was later solved by **RMSProp**, which we'll learn next.

---

10. Typical Hyperparameter Settings

Parameter	Typical Value
Learning Rate (η)	0.01 or 0.001
ε (epsilon)	1e-8
No momentum used	

11. Summary Table: SGD vs AdaGrad

Feature	SGD	AdaGrad
Learning Rate	Fixed	Adaptive (per parameter)
Gradient Memory	No	Yes (accumulates squared gradients)
Convergence	May zig-zag	More stable
Works Well For	Dense data	Sparse data
Problem	Oscillations	Learning rate decay too fast

12. Key Takeaways

1. AdaGrad = Adaptive Learning Rate Optimizer
2. Adjusts learning rate for each parameter individually
3. Helps with sparse features and high-dimensional data
4. May stop learning early due to shrinking learning rate
5. Basis for advanced optimizers like RMSProp and Adam

TL;DR (In One Line)

AdaGrad dynamically scales the learning rate for each parameter speeding up slow learners and slowing down fast ones but may eventually reduce learning too much.